BUT Science des Données 2 - Semestre 4

COMPETENCE 1 : Traiter des données à des fins décisionnelles

Ressource : **Programmation web (VCOD)**

F.GARNIER

PLAN DE LA RESSOURCE

Chapitre 1 : Le langage JAVASCRIPT

Chapitre 2 : Le framework javascript `Plotly`

Chapitre 3 : Le framework javascript `D3.js`

Chapitre 4 : Introduction à la librairie Python `streamlit`

Chapitre 5 : Découverte de la programmation web `Low-code`

Projet noté et/ou QCM

Chapitre 4 : streamlit

1. Introduction et préparation de l'environnement de travail

Nous allons étudier l'utilisation du package `streamlit`, équivalent Python de `shiny` pour R.

Le but de cette séance est de découvrir la librairie et ces premières possibilités, mais n'est pas d'être exhaustif sur celles-ci.

Nous utilisons donc ici l'outil VS Code. Suivre les étapes suivantes pour mettre en place les éléments nous permettant de créer, développer et tester notre application :

- Créer un nouveau répertoire dans l'explorateur Windows dans votre espace personnel (OneDrive ou autre)
- Dans VS Code, cliquer sur *File*, puis sur *Open Folder*
- Choisir le répertoire que vous venez de créer
- Créer un nouveau fichier en (bien choisir **Python File** en haut) :
    - Soit cliquant sur *New File* dans la fenêtre centrale
    - Soit allant sur *File* puis *New File…*
- Copier le code suivant dans le fichier :

```
import streamlit

streamlit.write("""
# Production scientifique mondiale
Voici un tableau de bord sur la **production scientifique mondiale**
                """)
```

Nous allons ensuite créer un environnement Python dans VS Code :

- Lancer la palette de commande (qui permet d'exécuter certaines opérations dans VS Code) en cliquant sur *View* puis *Command Palette*
- Chercher la commande *Python: Create environnement* (en écrivant la commande, il y a une sélection automatique des commandes listées)
- Cliquer ensuite sur *Venv*, puis choisir l'interpréteur par défaut

Un nouvel environnement est créé, visible car le dossier `.venv` est créé dans le répertoire de travail. Une fois celui-ci créé, vous devez l'activez.

- Lancer le *Terminal* en cliquant sur *Terminal* puis *New Terminal*
- Lancer la commande suivante : `.venv/Scripts/activate`

Nous allons maintenant installer la librairie `streamlit` et exécuter notre application pour la première fois : `pip install streamlit`

- Ensuite, lancer l'appli avec la commande suivante dans le Terminal : `streamlit run app.py`

Votre application doit se lancer dans le navigateur par défaut. Par la suite, vous pouvez **uniquement sauvegarder** votre fichier `app.py` (dans VS Code) **et recharger** (`Ctrl+R`) la page de l'appli (dans votre navigateur) à chaque modification.

2. Quelques éléments graphiques de base

Télécharger sur Updago le fichier `scimagojr.csv` dans le répertoire de votre application.

Affichage simple d'un dataframe

Ajouter le code suivant à votre application :

```
import pandas

data = pandas.read_csv("scimagojr.csv")
streamlit.write(data.query("Year == 2021").head(10))
```

Affichage plus spécifique

La fonction `dataframe()` est dédiée spécifiquement à l'affichage d'un dataframe pandas dans une application. Elle permet donc plus de possibilités.

On souhaite par exemple supprimer l'index des lignes :

```
streamlit.dataframe(data.query("Year == 2021").head(10), hide_index = True)
```

Ajout d'un slider (sélection d'une valeur quantitative discrète)

Sélection de l'année :

```
annee =
 streamlit.slider("Année", min(data["Year"]), max(data["Year"]), max(data["Year"]))

streamlit.dataframe(data.query("Year == @annee").head(10), hide_index = True)
```

Ajout de boutons radio (sélection d'une modalité parmi plusieurs)

Ici, on ajoute un choix de la taille du TOP (entre les valeurs 3, 5, 10 et 20)

```
taille = streamlit.radio("Taille", [3, 5, 10, 20], 1)

streamlit.dataframe(data.query("Year == @annee").head(taille), hide_index = True)
```

Si on souhaite avoir les boutons sur une ligne, on peut ajouter l'option `horizontal = True` à la fonction `radio()`

Ajout d'un sélecteur multiple

Pour le choix des régions, on veut laisser la possibilité d'en sélectionner plusieurs. Nous utilisons la fonction `multiselect()`. On doit gérer la possibilité qu'aucune sélection ne soit faite, en testant le

résultat de la sélection. Si celui-ci n'est pas vide (ce que fait la condition du `if`), il y a bien une sélection. Sinon, on affiche toutes les régions.

```
regions = streamlit.multiselect("Régions", set(data["Region"]))

if (regions):
    streamlit.dataframe(data.query("Year == @annee & Region in @regions").head(taille), hide_index = True)
else:
    streamlit.dataframe(data.query("Year == @annee").head(taille), hide_index = True)
```

Ajout d'une checkbox

On veut pouvoir alterner entre TOP (meillleurs) et FLOP (plus mauvais). Le plus simple est d'utiliser une checkbox.

A cette étape, il est préférable de ré-écrire le code pour le rendre plus lisible et que les opérations soient faites correctement :

```
data = pandas.read_csv("scimagojr.csv")

annee = streamlit.slider("Année", min(data["Year"]), max(data["Year"]), max(data["Year"]))
taille = streamlit.radio("Taille", [3, 5, 10, 20], 1, horizontal = True)
regions = streamlit.multiselect("Régions", set(data["Region"]))
flop = streamlit.checkbox("FLOP")

temp = data.query("Year == @annee")
if (regions):
    temp = temp.query("Region in @regions")
if (flop):
    temp = temp.iloc[::-1]

streamlit.dataframe(temp.head(taille), hide_index = True)
```

Un peu de mise en page en colonnes

On peut créer un système de colonnes pour répartir les sélecteurs par exemple. Ici, on créé deux colonnes, que l'on utilise pour les positionner autrement :

```
col1, col2 = streamlit.columns(2)

with col1:
    annee = streamlit.slider("Année", min(data["Year"]), max(data["Year"]), max(data["Year"]))
    taille = streamlit.radio("Taille", [3, 5, 10, 20], 1, horizontal = True)

with col2:
    regions = streamlit.multiselect("Régions", set(data["Region"]))
    flop = streamlit.checkbox("FLOP")
```

Ajout d'un titre

Après le tableau (ou après une figure), il est d'usage de mettre un titre, permettant de spécifier certaines informations (par exemple). Pour ajouter un tel titre (avec l'indication de TOP ou FLOP), on peut modifier le code comme ci-dessous :

```
msg = "TOP"
temp = data.query("Year == @annee")
if (regions):
    temp = temp.query("Region in @regions")
if (flop):
    temp = temp.iloc[::-1]
    msg = "FLOP"

streamlit.dataframe(temp.head(taille), hide_index = True)
streamlit.caption(msg + " des pays des régions sélectionnées (monde si aucune).")
```

3. Création d'un graphique

La librairie `streamlit` permet de faire un certain nombre de graphiques. On peut par exemple utiliser la fonction `line_chart()` pour voir l'évolution de la position des pays dans le TOP10 tout au long de la période.

```
streamlit.line_chart(data.query("Rank <= 10"), x = "Year", y = "Rank", color =
"Country")
```

Avec `plotly`

La librairie `plotly`, en plus d'être accessible en JS, l'est aussi en Python. En particulier, le sous-module `express` permet de réaliser des graphiques simplement.

Vous devez d'abord installer cette librairie : `pip install plotly`

```
import plotly.express as px

fig = px.line(data.query("Rank <= 10"), x = "Year", y = "Rank", color = "Country",
markers = True)
streamlit.plotly_chart(fig)
```

Si on veut pousser plus loin la création de ce graphique, le code devient plus complexe :

```
import plotly.graph_objects as go
fig = go.Figure()
for pays in set(data["Country"]):
    temp_pays = data.query("Country == @pays")
    temp_pays["rang"] = [e if e <= 10 else None for e in temp_pays["Rank"]]
    if any(temp_pays["rang"]):
        fig.add_trace(go.Scatter(
            x = temp_pays["Year"],
            y = temp_pays["rang"],
            mode = "lines+markers",
            name = pays
        ))
        temp_pays_fin = temp_pays.query("rang > 0").tail(1)
        ancreY = "middle"
        if list(temp_pays_fin["Year"])[0] < max(data["Year"]):
            ancreY = "top"
```

```
        fig.add_annotation(
            x = list(temp_pays_fin["Year"])[0], y = list(temp_pays_fin["rang"])[0],
            text = pays, showarrow = False, xanchor = "left", yanchor = ancreY,
xshift = 2)
fig.update_yaxes(range = [10.5,0.5])
fig.update_layout(showlegend = False)
streamlit.plotly_chart(fig)
```

4. Quelques éléments de texte

On dispose de plusieurs fonctions permettant d'écrire du texte et de le structurer :

- `write()` : pour intégrer qqch dans l'appli (résultat dépendant de ce qui est mis en paramètre)
- `text()` : pur texte
- `markdown()` : explicitement du markdown
- `title()` : titre de niveau 1
- `header()` : titre de niveau 2 (avec une ligne en dessous avec divider = True)
- `subheader()` : niveau 3
- `caption()` : titre d'une figure ou d'un tableau (plutôt à placer en dessous)
- `code()` : du code
- `divider()` : ligne de séparation

Dans le texte écrit en Markdown, on dispose aussi de différents éléments :

- des emojis possibles au format `":emoji:"`
    - Liste des émojis disponible ici https://share.streamlit.io/streamlit/emoji-shortcodes
- des icones Google Fonts au format `":material/nom_icone:"`
    - Liste disponible sur ce lien https://fonts.google.com/icons?icon.set=Material+Symbols&icon.style=Rounded
- de la couleur au format `":couleur[texte à colorier]"` (le texte) ou `":couleur-background[texte à surligner donc]"` (le fond)
    - Liste des couleurs usuelles HTML

Par exemple, ajouter ce texte dans l'application :

```
Voici un :red[**tableau de bord**] sur la :green-background[production scientifique
mondiale] :material/School: :sunglasses:
```

5. Structuration globale de l'application

Voici quelques éléments permettant de structurer (basiquement) une application.

Espace à gauche pour paramètres

A l'instar de `shiny`, il est courant d'avoir une barre sur le côté gauche regroupant certains éléments, comme des sélecteurs, ou un menu (ou d'autres éléments). On peut simplement créer un tel espace avec l'objet `sidebar` :

```
with streamlit.sidebar:
    annee = streamlit.slider("Année", min(data["Year"]), max(data["Year"]),
max(data["Year"]))
    taille = streamlit.radio("Taille", [3, 5, 10, 20], 1, horizontal = True)
    regions = streamlit.multiselect("Régions", set(data["Region"]))
    flop = streamlit.checkbox("FLOP")
```

Système d'onglets

Toujours comme dans `shiny`, on peut avoir besoin d'un système d'onglets, créés ici avec la fonction `tabs()` :

```
onglet1, onglet2 = streamlit.tabs(["TOP/FLOP", "Evolution TOP10"])

with onglet1:
    streamlit.header("TOP/FLOP de la production par année", divider = True)
    streamlit.dataframe(temp.head(taille), hide_index = True)
    streamlit.caption(msg + " des pays des régions sélectionnées (monde si aucune).")

with onglet2:
    streamlit.header("Evolution des rangs des pays du TOP10", divider = True)
    import plotly.express as px
    fig = px.line(data.query("Rank <= 10"), x = "Year", y = "Rank", color = "Country", markers = True)
    streamlit.plotly_chart(fig)
```

Noter qu'un système de menu et de pages est également possible.

Configuration globale

Enfin, il est possible de définir des paramètres globaux de l'application avec la fonction `set_page_config()`, avec les paramètres suivants :

- `page_title` : titre affiché dans le navigateur (pas sur la page donc)
- `page_icon` : icône affiché dans le navigateur
- `layout` : l'option `wide` permet d'élargir la zone dans laquelle l'application est présentée

```
streamlit.set_page_config(
    page_title = "Production scientifique mondiale | Séance 6",
    page_icon = ":student:",
    layout = "wide"
)
```

6. Gestion des sélections (et plus généralement des évènements)

Au final, puisque nous avons créé une application web, il est important de gérer les interactions avec l'utilisateur. Cela passera forcément par les gestions des sélections dans un tableau et/ou un graphique. Voici deux exemples :

Dans un dataframe

On affiche un dataframe avec la fonction `dataframe()` donc, avec le résultat stocké dans une variable. Ce résultat sera la sélection (s'il y en a une) de l'utilisateur. Les lignes en-dessous de `selection = streamlit.dataframe(...)` permettent justement de voir le contenu de celle-ci.

Vous pouvez observer l'usage des paramètres suivants dans la fonction `dataframe()` :

- `selection_mode` : définit ce qu'il est possible de sélectionner (ici, plusieurs lignes via `"multi-row"`)
- `on_select` : définit ce que doit faire l'appli une fois la sélection faite (ici, on la redémarre avec `"rerun"`)

```
streamlit.set_page_config(layout="wide")
data = pandas.read_csv("scimagojr.csv")
top10 = data.query("Year == 2021").head(10)
col1, col2 = streamlit.columns(2)

liste_pays = []

with col1:
    selection = streamlit.dataframe(top10, hide_index = True, selection_mode = "multi-row", on_select = "rerun")
    streamlit.write(selection)
    streamlit.write(selection["selection"]["rows"])
    if (selection["selection"]["rows"]):
        streamlit.write(top10.iloc[selection["selection"]["rows"]])
        liste_pays = list(top10.iloc[selection["selection"]["rows"]]["Country"])
        streamlit.write(liste_pays)

with col2:
    if (liste_pays):
        fig = px.line(data.query("Rank <= 10 & Country in @liste_pays"), x = "Year", y = "Rank", color = "Country", markers = True)
    else:
        fig = px.line(data.query("Rank <= 10"), x = "Year", y = "Rank", color = "Country", markers = True)
    streamlit.plotly_chart(fig)
```

Noter la gestion de la sélection pour ne voir afficher sur le graphique que les pays choisis (s'il y en a).

Dans un diagramme en barres `plotly`

L'autre opération est un peu *l'inverse* : on sélectionne dans un graphique `plotly` (en laissant la possibilité de ne sélectionner un seul objet – une barre ici – que via un clic simple – valeur `"points"`).

```
streamlit.set_page_config(layout="wide")

data = pandas.read_csv("scimagojr.csv").query("Year == 2021").sort_values(by = "H index", ascending = False).head(20)

col1, col2 = streamlit.columns(2)

with col1:
    fig = px.bar(
        data.iloc[::-1],
        y = "Country",
        x = "H index"
    )
    selection = streamlit.plotly_chart(fig, on_select = "rerun", selection_mode = "points")
    streamlit.write(selection)

with col2:
    temp = data.filter(["Country", "Region", "H index", "Documents"])
    if (selection["selection"]["points"]):
        indice_pays = 19 - selection["selection"]["points"][0]["point_index"]
        temp = temp.style.applymap(lambda _: "background-color: lightblue", subset=(temp.index[indice_pays],))
    streamlit.dataframe(temp, hide_index = True, height = 750, use_container_width = True)
```

Noter la gestion de la sélection pour mettre en valeur le pays choisi (via une couleur de fond)